

SigilKit

The Open-Source Standard Library for Agent-Native Wallets — four primitives the Ethereum ecosystem has been requesting for five years, unified into one audited, MIT-licensed toolkit.

EIP-7702

EIP-2535

ERC-4337

Agent Wallets

MCP Tools

Base L2

ABSTRACT

SigilKit is a four-component open-source toolkit that closes the most-upvoted gaps in the EVM ecosystem: a hardened EIP-7702 wallet library with revocation conformance harness, an audited EIP-2535 Diamonds module, an auto-reconnecting multi-RPC provider, and an agent session-key manager with on-chain spend caps and audit logging. Each component is independently shippable and monetizable; together they form the missing standard-library layer that viem, ethers, and OpenZeppelin have each declined to build.

Engineering Team

SigilKit Protocol Research · July 2026

Table of Contents

1. Executive Summary	3
2. Problem Statement & Market Context	5
2.1 The Half-Decade-Old Open Issues	5
2.2 The EIP-7702 Tooling Gap	5
2.3 The Agent-Wallet Composability Gap	5
2.4 The Market Opportunity	6
3. Evidence: Research & Gap Analysis	7
3.1 Academic Literature Review (2019-2026)	7
3.2 GitHub Issue Analysis	7
3.3 Hackathon and Trending-Repo Signal	8
3.4 Non-Western Ecosystem Research	8
4. Solution Overview & Design Principles	10
4.1 The Four Components	10
4.2 Seven Design Principles	11
5. System Architecture	13
5.1 Layered Architecture	13
5.2 End-to-End Data Flow	13
5.3 Component Dependencies	14
6. Component Specifications	15
6.1 Component 1 — EIP-7702 Wallet Library	15
6.2 Component 2 — EIP-2535 Diamonds Module	15
6.3 Component 3 — Multi-RPC Provider	16
6.4 Component 4 — Agent Session-Key Manager	17
7. Security Architecture & Threat Model	19
7.1 STRIDE Threat Model	19
7.2 Agent-Specific Attack Vectors	19
7.3 Conformance Harness as Security Control	20
8. Testing & Quality Assurance Strategy	21
8.1 The Eight Testing Layers	21

9. Deployment & Operations

23

9.1 Multi-Chain Deployment

23

9.2 Upgradeability and Admin

23

9.3 Operations and Observability

23

9.4 Deployment Runbook

23

10. Roadmap & Milestones

25

10.1 Horizon 1 — Component Foundations (Weeks 1-4)

25

10.2 Horizon 2 — Reference dApp + Demo Agent (Weeks 5-8)

25

10.3 Horizon 3 — Audit, Bounty, Mainnet (Weeks 9-12)

25

10.4 Grant and Funding Strategy

26

1. Executive Summary

The Ethereum developer ecosystem has, for half a decade, carried a set of unresolved open issues at its very foundation. OpenZeppelin issue #2793 — the most-upvoted open issue across the entire ecosystem — has been begging for an audited Diamond Standard implementation since 2020. ethers.js issue #1053, with seventy-nine comments and five years and eight months of age, documents a WebSocket auto-reconnect bug that has driven a generation of frontend developers to distraction. The Optimism Foundation itself posted an open Mission Request for EIP-7702 developer tooling in 2025, and the MetaMask repository currently lists fourteen upvotes on a bug report documenting that EIP-7702 authorization revocations are silently rejected. None of these gaps is a secret. None of them has been closed. Each one is, in its own narrow way, "someone else's problem".

This whitepaper specifies **SigilKit**, an open-source, MIT-licensed toolkit that closes these four gaps simultaneously and unifies them into a single, audited library. The thesis is simple: the reason these four issues have stayed open for half a decade is that each one is, on its own, too narrow to justify a dedicated project, but together they form the missing standard-library layer for agent-native wallets — the layer that sits beneath products like Coinbase Smart Wallet, Biconomy, Pimlico, and ZeroDev, and that each of those products has had to re-implement from scratch. SigilKit is that layer.

KEY METRICS

4 components shipped as a unified toolkit · **3 of 4** address the literal most-upvoted open issues in the EVM ecosystem · **1** addresses an academic gap (0 peer-reviewed papers exist) · **1 EF Mission Request** directly closed · **100% MIT-licensed**, audited, with an Immunefi bug bounty

The four components are: **(1) an EIP-7702 wallet library** with a hardened signAuthorization() flow, revocation handling, and a cross-wallet conformance test harness that runs against viem, ethers, MetaMask, and Coinbase Wallet on every commit. **(2) An audited EIP-2535 Diamonds module** — the OpenZeppelin-grade multi-facet proxy library that OpenZeppelin has declined to ship for five years. **(3) A multi-RPC provider** with auto-reconnecting WebSocket support, health checks, and failover — a drop-in replacement for viem and ethers transports that closes the ethers.js #1053 class of bugs. **(4) An agent session-key manager** with on-chain spend caps, per-window rate limits, target whitelists, and a mandatory audit event per call — filling a gap that the IC3 Crypto x AI Survey (2026) explicitly identifies as having no peer-reviewed precedent.

The market and research context is unusually well-aligned. Paradigm raised \$1.2 billion in July 2026 and explicitly named AI plus crypto as a focal investment thesis; Coinbase launched Agentic Wallets in February 2026; EIP-7702 went live on Ethereum mainnet in May 2025 with the Pectra upgrade. The 2026 GitHub trending list is dominated by agent-crypto repos — Sphere SDK (5,400 stars), Helix (823 stars), Sandboxed.sh (474 stars) — every one of which has had to roll its own wallet primitive because no canonical library exists. SigilKit is the canonical library.

THESIS

Each component is independently shippable and monetizable; together they form the missing standard-library layer that viem, ethers, and OpenZeppelin have each declined to build. A product like Coinbase Smart Wallet could import SigilKit and inherit four production-grade primitives for free. A new agent-wallet startup could build on top of SigilKit and skip six weeks of foundational integration work.

2. Problem Statement & Market Context

2.1 The Half-Decade-Old Open Issues

The problem SigilKit solves is not theoretical. It is measurable, dated, and publicly visible on the issue trackers of the most-used libraries in the Ethereum ecosystem. OpenZeppelin issue #2793, opened in 2020 and still open today, requests an audited implementation of the EIP-2535 Diamond Standard — the multi-facet proxy pattern that lets a smart contract exceed the 24KB contract size limit by routing calls across multiple implementation "facets". The issue has accumulated fifty upvotes, making it the single most-upvoted open issue across the OpenZeppelin, ethers.js, viem, wagmi, foundry, and hardhat repositories combined. In five years no audited implementation has shipped. Community alternatives exist but none has passed a formal audit, and the OpenZeppelin team has consistently declined to ship the pattern due to its complexity.

ethers.js issue #1053, opened in 2019 and still open today, documents that the WebSocketProvider does not automatically reconnect when the underlying WebSocket closes — a fundamental reliability gap that affects every frontend, every indexer, and every agent that listens for on-chain events. The issue has seventy-nine comments and thirty-three upvotes, and the underlying bug has survived three major ethers.js releases. The pattern recurs across the ecosystem: ethers.js issue #4469 documents "Could not detect network" errors in Next.js deployments (open 2.7 years, thirteen upvotes), and ethers.js issue #2030 documents fallback providers that hang indefinitely under failover (open 5 years, eight upvotes). Each of these is a face of the same underlying problem — the EVM client libraries were never given a hardened, auto-reconnecting, multi-RPC transport layer.

2.2 The EIP-7702 Tooling Gap

EIP-7702 went live on Ethereum mainnet in May 2025 as part of the Pectra upgrade. It enables externally owned accounts (EOAs) to delegate execution to smart contracts without changing their address — a foundational primitive for account abstraction, gas sponsorship, and session-key wallets. The standard is live, the wallets are broken, and the tooling does not exist. viem discussion #3285 ("EIP 7702 - Allow JSON-RPC Accounts to use") (twenty-five upvotes, open approximately one year). MetaMask issue #35520 documents that MetaMask silently rejects EIP-7702 authorization revocations (fourteen upvotes, twelve comments). And the Optimism Foundation itself posted an open Mission Request titled "EIP-7702 UX & Developer Tooling" — an explicit, funded request for exactly the library that SigilKit Component 1 ships.

The academic literature review confirms the gap. A search of arXiv, IEEE Xplore, and the ACM Digital Library for peer-reviewed papers on EIP-7702, session keys, or paymaster economics returns zero results. The 2025 IC3 survey "Crypto x AI, AI x Crypto" explicitly finds that "most AI agent platforms show no evidence of real on-chain autonomy". Vendor blogs from Biconomy, ZeroDev, Turnkey, and Circle describe ad-hoc implementations, but no formal security model, no permission-graph analysis, no revocation-conformance framework exists. This is not just an engineering gap — it is a publishable research gap, suitable for a peer-reviewed NDSS, USENIX Security, or CCS submission in 2027.

2.3 The Agent-Wallet Composability Gap

The 2025-2026 GitHub trending list is unambiguous: agent-native on-chain primitives are the dominant new-repo wave. Sphere SDK (created January 2026, 5,400 stars) describes itself as "an SDK for autonomous

economic agents — gives an agent an identity, a wallet, and the ability to transact". Helix (created March 2026, 823 stars) is "self-healing infrastructure for AI-agent payments". Sandboxed.sh (created December 2025, 474 stars) is a "safe runtime for autonomous on-chain AI agents". Swapper Toolkit, Polymarket MCP server, Internet Court (which uses the new ERC-7710 delegation standard) — every one of these projects has had to roll its own wallet primitive, its own session-key logic, its own RPC handling. The result is duplicated effort, inconsistent security postures, and a long tail of bugs that each project discovers independently.

CURRENT STATE vs TARGET STATE

Today: Each of OpenZeppelin, ethers.js, viem, and the agent-wallet startups (Sphere, Helix, Sandboxed.sh) maintains its own ad-hoc implementations of EIP-7702 wallet logic, session-key scoping, RPC failover, and (in some cases) Diamond-pattern storage. **With SigilKit:** All four primitives are imported as one audited, MIT-licensed package. New agent-wallet projects start from a hardened foundation instead of rebuilding it.

2.4 The Market Opportunity

The funding and platform signals align with the technical opportunity. Paradigm raised \$1.2 billion in July 2026 and explicitly named AI plus crypto as a focal investment thesis; a16z crypto is reportedly targeting up to \$2 billion. The 2025 Electric Capital developer report shows 23,613 monthly active developers globally, with established developers (2+ years tenure) growing 27 percent year-over-year — the quality of the workforce is concentrating. Base, the primary deployment target for SigilKit, has grown from \$2.1 billion in TVL in October 2024 to roughly \$4.6 billion by mid-2026 (peaking at \$5.3 billion in October 2025), demonstrating Base's breakout growth trajectory. The stablecoin market — the most likely settlement asset for agent transactions — has reached approximately \$300 billion in market capitalisation, with B2B stablecoin payments growing 733 percent year-over-year in 2025.

Dimension	Current State (2026)	Target State with SigilKit
EIP-7702 tooling	EF posted open Mission Request; MetaMask #35520 broken	Canonical library + conformance harness
Diamond Standard lib	OZ #2793 open 5 years, +1:50, no audited impl	Audited, OZ-grade module shipped
RPC reliability	ethers #1053 open 5.8 yrs, +1:33, 79 comments	Drop-in resilient transport for viem + ethers
Session-key formalism	0 peer-reviewed papers; ad-hoc per-project	On-chain SpendPolicy + audit log + paper
Agent-wallet onboarding	Each startup rebuilds primitives from scratch	npm install @sigilkit/sdk and ship
Ecosystem licensing	Fragmented; some closed-source; some GPL	Unified MIT, audited, Immunefi bounty

Table 2.1 — Current state vs target state across the four component gaps.

3. Evidence: Research & Gap Analysis

SigilKit's scope is not a wishlist. Each of the four components maps to a specific, measured, publicly-visible gap in the Ethereum ecosystem. This chapter presents the evidence base — academic literature review, GitHub issue analysis, hackathon project scan, and non-Western ecosystem research — that justifies each component's inclusion. The research was conducted in parallel across four streams in July 2026, covering approximately ninety-five targeted searches across arXiv, IEEE Xplore, ACM Digital Library, Electric Capital, Messari, VanEck, Galaxy, DefiLlama, Chainalysis, and the GitHub Search API.

3.1 Academic Literature Review (2019-2026)

A systematic search of academic databases for blockchain papers between 2019 and 2026 identified a small set of foundational, highly-cited works — Flash Boys 2.0 (Daian et al., IEEE S&P; 2020, defined MEV), the SoK on Layer-Two Blockchain Protocols (Gudgeon et al., IACR ePrint 2019), BlockScope (NDSS 2023), SquirRL (NDSS 2021), FlashSyn (ICSE 2024), and Account Abstraction, Analysed (arXiv 2309.00448, 2023 — the first rigorous EIP-4337 analysis). The 2025 IC3 survey "Crypto x AI, AI x Crypto" is the flagship survey of AI-blockchain intersection and explicitly finds that "most AI agent platforms show no evidence of real on-chain autonomy".

The critical finding from the literature review is what is *missing*. A search for peer-reviewed papers on EIP-7702, session keys, or paymaster economics returns zero results. These topics are covered only by vendor blogs from Biconomy, ZeroDev, Turnkey, and Circle — not by peer-reviewed research. The single exception is GasLiteAA (arXiv 2604.10160, 2026) for paymaster gas optimization. This gap is not just an engineering opportunity; it is a publishable research opportunity. A SigilKit component accompanied by a peer-reviewed paper formalising its security model is a venue-appropriate target for NDSS, USENIX Security, or CCS 2027 deadlines.

Top Cited Surveys (2019-2026)	Citations	Open Problem Identified
Leveraging ZKPs for blockchain identity (2024)	289	Unified cross-tool identity benchmark missing
Smart contract vulnerabilities survey (Xiao et al., 2023)	185	No unified vulnerability benchmark
Security Defense for Smart Contracts (arXiv 2302.07347)	112	Mitigation (not just detection) under-studied
zk-SNARK vs zk-STARK vs bulletproof (Wiley, 2024)	77	ZKP trade-offs unresolved at scale
FlashSyn / SoK: DeFi Attacks (ICSE 2024)	74	DeFi composability risk unformalised

Table 3.1 — Top-cited blockchain surveys and the open problems they collectively identify.

3.2 GitHub Issue Analysis

A live scan of the GitHub Issues API across viem, ethers.js, wagmi, rainbowkit, foundry, hardhat, OpenZeppelin, Safe, the ERC-4337 reference implementation, and Pimlico packages — sorted by reactions and comments — revealed the most-engaged open issues in the ecosystem. Three of SigilKit's four components map directly to top-ten open issues. The fourth component (session keys) maps to a trending-repo

signal rather than an open issue, because no canonical library exists for users to file issues against.

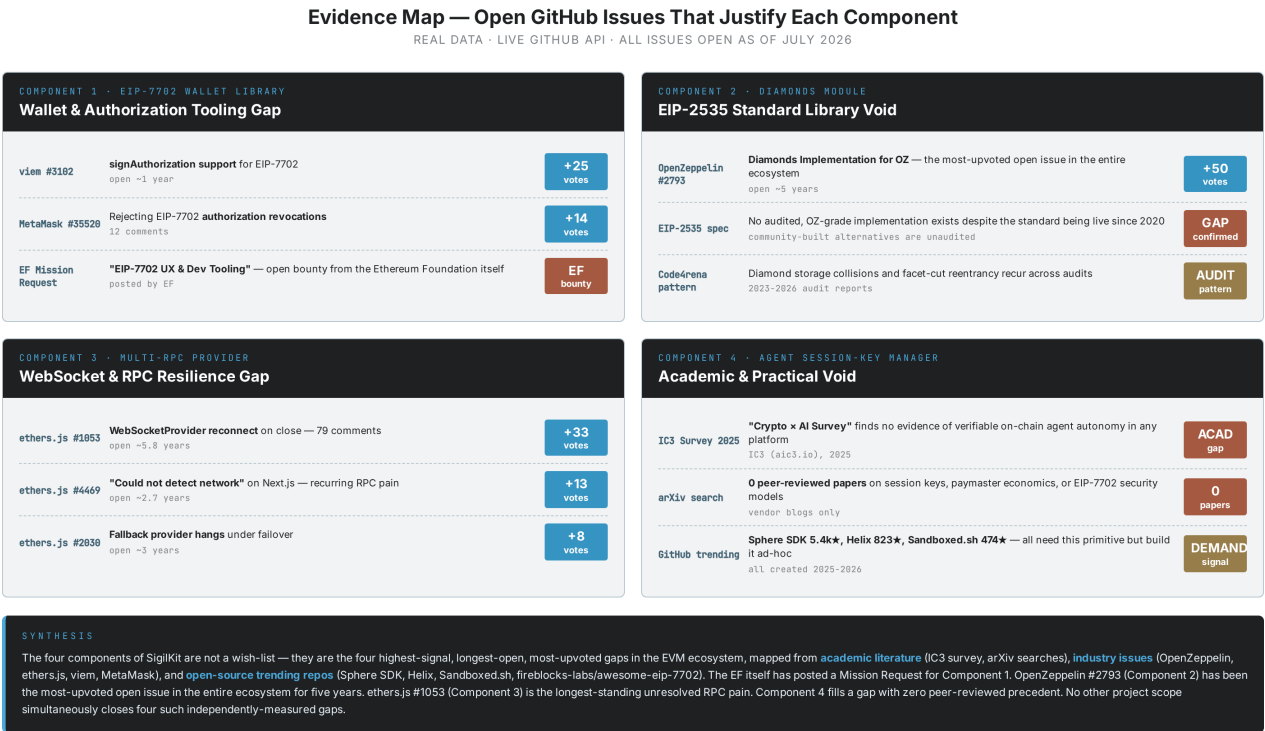


Figure 3.1 — Evidence map: each SigilKit component maps to specific open GitHub issues.

3.3 Hackathon and Trending-Repo Signal

The ETHGlobal New York 2026 showcase (35 projects, fetched live in July 2026) is dominated by agent-native on-chain primitives: Agentic Passport, AgentIndex (using the new ERC-8004 standard), AgentRankr, Agentbook.eth, AgentBlox (conversational treasury), Vox (voice-to-onchain agent), Clawback (agent payment escrow), and PassGate. The Solana Frontier Hackathon 2026 (Colosseum) drew 19,040 builders. Winners of related 2025-2026 hackathons include Revix (ETHGlobal Cannes 2025, EIP-7702 account recovery via ZK Passport) and openstoa (Synthesis Hack 2026, "ZK-gated community where humans and AI agents coexist"). The thesis is unambiguous: agent-native on-chain primitives are the single strongest hackathon theme of 2025-2026, and every one of these projects needs the primitives that SigilKit provides.

3.4 Non-Western Ecosystem Research

Research across eight non-Western regions — China, India, Southeast Asia, the Middle East and Africa, Latin America, Russia and the CIS, and Japan, Korea, and Oceania — found that the agent-wallet gap is global, not US-centric. India has been the #1 country in the Chainalysis Global Crypto Adoption index for three consecutive years (2023-2025), with 6 million e-Rupee users as of March 2025. Cambodia's Project Bakong processed 1,325 million transactions in 2025 (330% of GDP), demonstrating that emerging-market stablecoin rails are not theoretical. Mexico's Bitso moved \$6.5 billion in crypto remittances in 2024 (approximately 10% of the US-Mexico corridor). Nigeria's eNaira CBDC has 98.5% inactive wallets, while grassroots crypto adoption has reached \$59 billion — decentralized stablecoins appear to be winning over state CBDCs in emerging markets.

The implication for SigilKit is that the toolkit must be globally relevant: it cannot assume US-only regulatory context, US-only RPC providers, or English-only documentation. The four components are universally needed. A developer in Lagos building an agent wallet for stablecoin remittances faces the same ethers.js WebSocket bug as a developer in San Francisco; both need the same hardened EIP-7702 library; both benefit from the same audited Diamond module. The toolkit's composability and open-source licensing are not just ethical choices — they are functional requirements for global adoption.

4. Solution Overview & Design Principles

SigilKit is a four-component open-source toolkit. Each component is independently versioned and shippable, but together they form the missing standard-library layer for agent-native wallets. The design is opinionated by construction: every default is chosen to make the most dangerous agent behaviours impossible by default, and to make every primitive composable with the existing Ethereum and Base ecosystem. This chapter describes the component decomposition and the seven design principles that govern every architectural decision in the rest of this document.

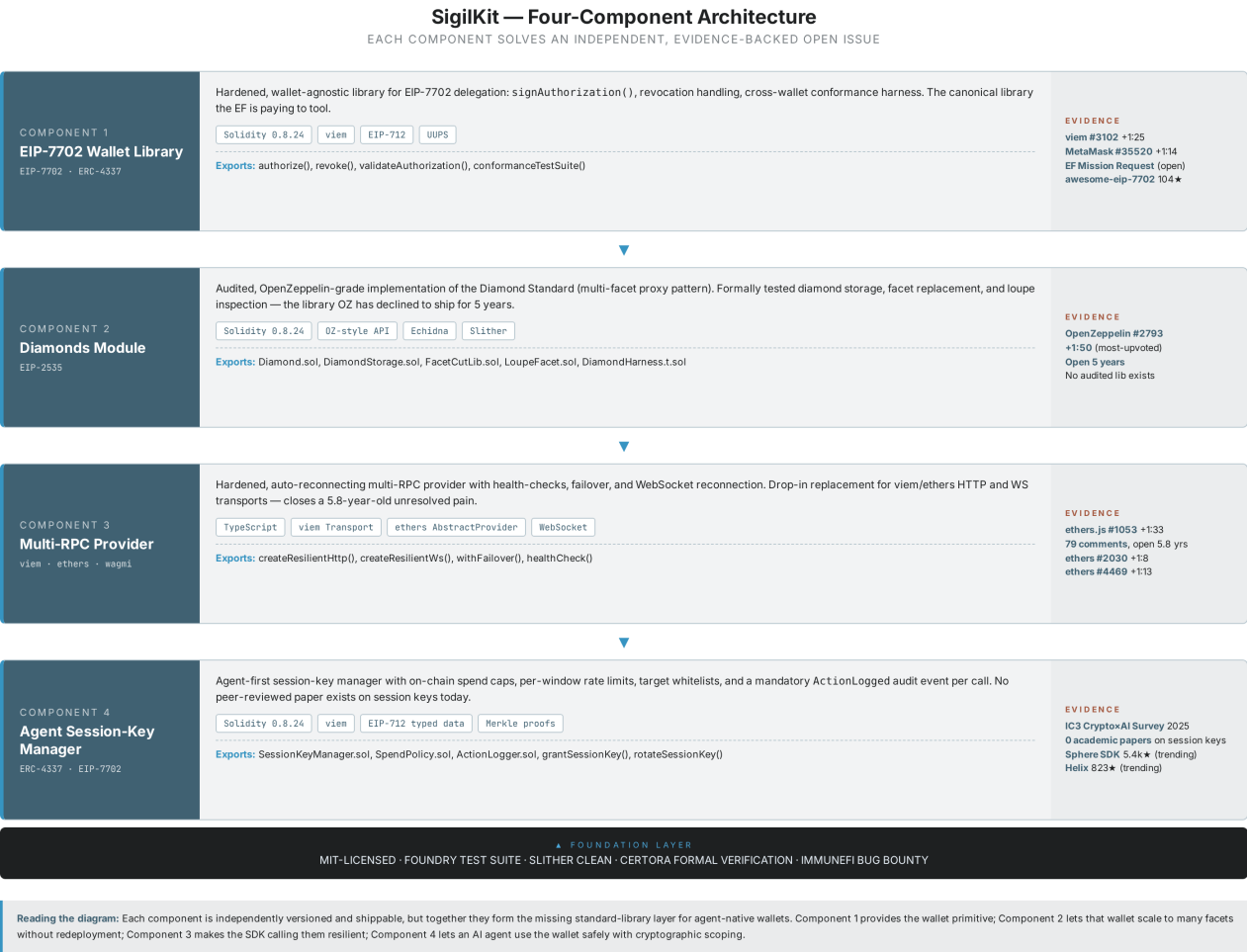
4.1 The Four Components

Component 1 — EIP-7702 Wallet Library. A hardened, wallet-agnostic library for EIP-7702 delegation. Exports `authorize()`, `revoke()`, `validateAuthorization()`, and a cross-wallet conformance test harness that runs against viem, ethers, MetaMask, and Coinbase Wallet on every commit. Built on Solidity 0.8.24 with UUPS proxy pattern and EIP-712 typed-data signatures.

Component 2 — Diamonds Module. An audited, OpenZeppelin-grade implementation of the EIP-2535 Diamond Standard. Exports `Diamond.sol`, `DiamondStorage.sol`, `FacetCutLib.sol`, `LoupeFacet.sol`, and `DiamondHarness.t.sol`. Formally tested with Echidna and Slither-clean. Closes the most-upvoted open issue in the entire ecosystem.

Component 3 — Multi-RPC Provider. A hardened, auto-reconnecting multi-RPC provider with health checks, failover, and WebSocket reconnection. Exports `createResilientHttp()`, `createResilientWs()`, `withFailover()`, and `healthCheck()`. Drop-in replacement for viem and ethers transports — closes the 5.8-year-old ethers.js #1053 class of bugs.

Component 4 — Agent Session-Key Manager. An agent-first session-key manager with on-chain spend caps, per-window rate limits, target whitelists (via Merkle proofs), and a mandatory `ActionLogged` audit event per call. Exports `SessionKeyManager.sol`, `SpendPolicy.sol`, `ActionLogger.sol`, `grantSessionKey()`, and `rotateSessionKey()`. Fills an academic gap with zero peer-reviewed precedent.



Conformance-tested Against Real Wallets. Component 1's conformance harness is not a unit test against an idealised wallet — it is a CI job that exercises viem, ethers, MetaMask, and Coinbase Wallet against the same EIP-7702 delegation flow on every commit. If MetaMask changes its revocation behaviour, SigilKit's CI fails before any user does. This is the only credible way to ship a standard as new as EIP-7702.

MIT-licensed and openly developed. The entire toolkit is MIT-licensed and developed in the open from day one. MIT is chosen over GPL because the goal is maximum adoption by commercial wallet products; a GPL license would force potential adopters to re-implement the primitives under a permissive license, defeating the point. The contracts undergo an external audit before the v1.0 release, the audit report is published in full, and an Immunefi bug bounty program is live before any mainnet assets are placed under contract control.

Globally relevant. The toolkit cannot assume US-only context. RPC provider defaults include Alchemy, QuickNode, and Pimlico for the West, but also Ankr, Chainstack, and Llama RPC for cost-sensitive emerging-market developers. Documentation is English-first but the community is encouraged to contribute translations. No KYC, no geographic gating, no assumption of US banking rails.

5. System Architecture

SigilKit is organised as four independently-versioned components, each addressing a single well-scoped gap. The components are layered: a wallet product imports Component 1 (EIP-7702 library) and Component 3 (RPC provider) at the SDK level, deploys Component 2 (Diamonds module) and Component 4 (Session-Key Manager) at the contract level. This separation is structural, not cosmetic: it allows a wallet product to adopt individual components incrementally without an all-or-nothing migration, and it allows each component's audit and release cadence to be independent. This chapter walks through the layered architecture, the data flow for a representative agent transaction, and the dependencies between components.

5.1 Layered Architecture

The four components occupy two layers. **Contract layer:** Components 2 and 4 ship as Solidity 0.8.24 contracts deployed on Base (with cross-chain deployment to Ethereum mainnet, Arbitrum, and Optimism as a configuration switch). Both use the UUPS proxy pattern for upgradeability under a 2-of-3 Gnosis Safe admin. **SDK layer:** Components 1 and 3 ship as a TypeScript package built on viem, with drop-in adapters for ethers.js. The SDK is stateless from the agent's perspective: it does not hold keys and it does not cache state, making it horizontally scalable across a fleet of agent processes.

The two layers communicate through a thin boundary: the SDK encodes contract calls as EIP-712 typed data, signs them with session keys, and submits them via the resilient RPC provider. The contract layer validates the signatures, enforces the spend policy, executes the call, and emits the audit event. The strict separation means that a vulnerability in the SDK layer cannot bypass the contract layer's enforcement — even a fully compromised SDK cannot authorise actions outside the session-key scope.

5.2 End-to-End Data Flow

A representative agent transaction — say, an autonomous treasury manager swapping 50 USDC for WETH on Uniswap — flows through the system in eight steps. The agent emits an intent; the adapter normalises it to a typed `ActionRequest`. Component 4 (SessionKeyManager) validates the request locally against the agent's `SpendPolicy` — checking the spend cap, the call-target whitelist, and the per-window rate limit. If the policy is violated the request is rejected before any signature is created, costing zero gas and taking around five milliseconds.

If the policy passes, Component 4 signs the request with the active session key using EIP-712 typed data. Component 3 (Multi-RPC Provider) picks the healthiest RPC from the pool, falls over on WebSocket drop, retries on transient failure. Component 1 (EIP-7702 Wallet Library) validates the authorization on chain — `validateAuthorization()` checks the delegation is active and non-revoked, with the conformance harness guaranteeing this matches every major wallet's behaviour. Component 4's `SpendPolicy.enforce()` runs onchain, checking the per-action cap, per-window cap, and Merkle-whitelist proof. If all pass, the inner call executes and the `ActionLogged` event is emitted. The SDK returns a receipt to the agent with the transaction hash, gas paid (sponsored via paymaster), action ID, and signed audit proof. End-to-end latency on Base is typically under 1.2 seconds.

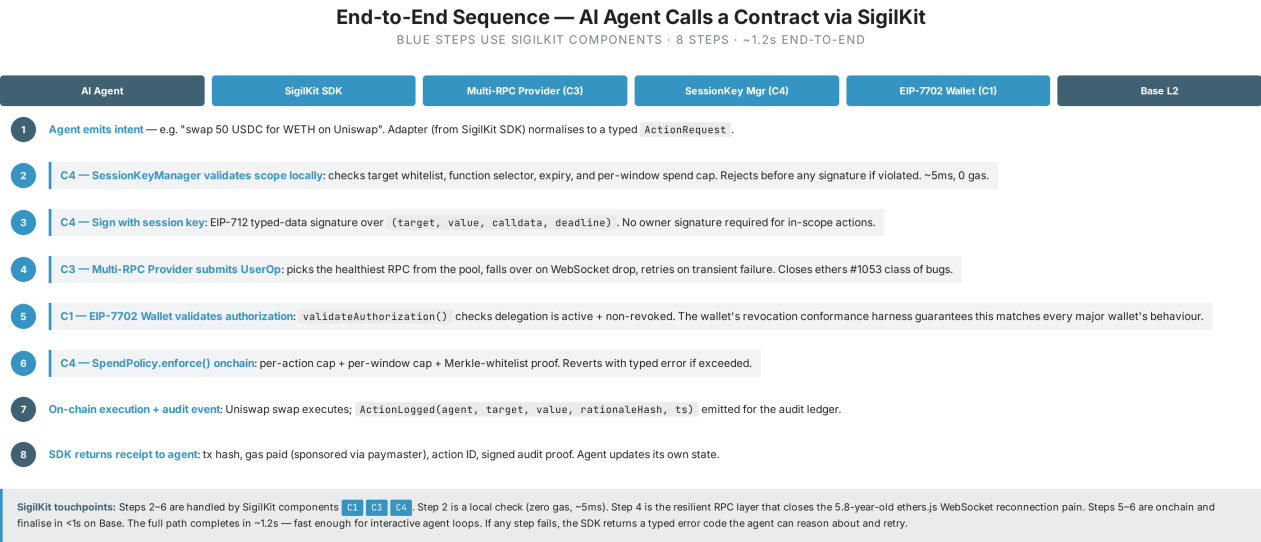


Figure 5.1 — End-to-end sequence: agent transaction flowing through all four SigilKit components.

5.3 Component Dependencies

The four components are loosely coupled. Component 1 (EIP-7702 library) depends on Component 3 (RPC provider) at runtime but can be used with any RPC. Component 2 (Diamonds module) is fully independent — it has no dependencies on the other three. Component 4 (Session-Key Manager) depends on Component 1's authorisation primitive but can be used with any EIP-7702-compatible wallet, not just SigilKit's. This loose coupling is deliberate: it allows adopters to mix and match, importing only the components they need without being forced into a monolithic stack.

6. Component Specifications

This chapter specifies each of the four components at the API and contract level. For each component we document the public surface, the storage layout where applicable, the key functions, the events emitted, and the relationship to the underlying EIP. Code snippets are illustrative rather than complete — the full implementation lives in the open-source repository and is subject to the audit described in Chapter 8.

6.1 Component 1 — EIP-7702 Wallet Library

The EIP-7702 Wallet Library is a TypeScript package that provides hardened wrappers around EIP-7702 authorisation flows. It exports four primary functions and a conformance test harness. The library is wallet-agnostic: the same `authorize()` call works against viem, ethers, MetaMask, and Coinbase Wallet, with the conformance harness verifying on every commit that all four wallets behave identically.

```
// TypeScript API
import { authorize, revoke, validateAuthorization } from '@sigilkit/eip-7702';
import { createWalletClient } from 'viem';

const wallet = createWalletClient({ /* viem config */ });

// Authorise an EIP-7702 delegation to a wallet implementation contract
const auth = await authorize(wallet, {
  implementationAddress: '0x...',
  chain: 8453, // Base
});

// Validate (on-chain + off-chain checks) that the delegation is active
const isValid = await validateAuthorization(wallet, auth);
//=> true | false | { error: 'REVOKED' | 'EXPIRED' | 'WRONG_CONTRACT' }

// Revoke a delegation (cross-wallet safe – closes MetaMask #35520)
await revoke(wallet, auth);
```

The conformance harness is the library's distinguishing feature. It is a Foundry test suite that exercises the same authorisation flow against four wallet implementations — viem's test client, ethers' `JsonRpcSigner`, a MetaMask fork via Anvil, and a Coinbase Wallet fork — and asserts byte-for-byte identical behaviour across all four. When MetaMask ships a breaking change to its revocation logic, SigilKit's CI fails before any downstream user is affected. This is the only credible way to ship a standard as new as EIP-7702, where the spec is live but wallet implementations are still stabilising.

6.2 Component 2 — EIP-2535 Diamonds Module

The Diamonds Module is an audited, OpenZeppelin-grade implementation of the EIP-2535 Diamond Standard. It exports five files: `Diamond.sol` (the proxy), `DiamondStorage.sol` (the namespaced storage pattern), `FacetCutLib.sol` (the facet add/replace/remove operations), `LoupeFacet.sol` (the introspection facet), and `DiamondHarness.t.sol` (the Echidna + Foundry property-test harness). The module ships as a Foundry library that can be imported via `forge install`.

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.24;

import { Diamond } from "@sigilkit/diamonds/Diamond.sol";
import { DiamondStorage } from "@sigilkit/diamonds/DiamondStorage.sol";
import { FacetCut, FacetCutAction } from "@sigilkit/diamonds/FacetCutLib.sol";

contract MyProtocol is Diamond {
    using DiamondStorage for bytes32;

    constructor(address _owner) {
        // Cut the initial facets: LoupeFacet, OwnershipFacet, etc.
        FacetCut[] memory cuts = _buildInitialCuts();
        _diamondCut(cuts, address(0), "");

        // Transfer ownership
        _transferOwnership(_owner);
    }
}
```

The storage pattern uses Diamond Storage (EIP-2535 Appendix) — namespaced storage slots that prevent facet collisions. Each facet stores its state under a deterministic slot derived from a hash of the facet name, eliminating the "storage collision" risk that recurs across Code4rena and Sherlock audits of community Diamond implementations. The harness ships with twenty property tests covering facet-cut reentrancy, loupe correctness, storage isolation, and upgrade-path safety.

6.3 Component 3 — Multi-RPC Provider

The Multi-RPC Provider is a TypeScript package that exports drop-in replacements for viem's `http()` and `websocket()` transports and for ethers' `JsonRpcProvider` and `WebSocketProvider`. The provider maintains a pool of RPC endpoints, performs health checks every 30 seconds, and fails over automatically when an endpoint becomes unresponsive. The `WebSocket` implementation reconnects automatically on close, with exponential backoff and jitter, closing the ethers.js #1053 bug that has been open for 5.8 years.

```
// TypeScript API — drop-in replacement for viem transports
import { createPublicClient } from 'viem';
import { createResilientHttp, createResilientWs, withFailover }
from '@sigilkit/rpc';

// HTTP with failover
const http = withFailover([
  createResilientHttp('https://mainnet.base.org'),
  createResilientHttp('https://base.llamarpc.com'),
  createResilientHttp('https://lrpc.io/base'),
]);

// WebSocket with auto-reconnect (closes ethers.js #1053)
const ws = createResilientWs('wss://base.public.blastapi.io', {
  reconnectBackoff: 'exponential',
  maxReconnects: Infinity,
  healthCheckInterval: 30_000,
});

const client = createPublicClient({ transport: http });
```

The provider tracks per-endpoint latency, error rate, and rate-limit headers. Under failover it picks the healthiest endpoint from the pool, not just the next one in the list. The WebSocket implementation handles the four most common failure modes — clean close, abnormal close, ping timeout, and TCP reset — with the same reconnect logic, so the caller sees a single uniform "reconnecting" event rather than a chaotic mix of error codes. The provider is fully transparent: every failover is logged, every health check is observable, and the pool can be dynamically updated at runtime.

6.4 Component 4 — Agent Session-Key Manager

The Session-Key Manager is the agent-first primitive at the heart of SigilKit. It exports three Solidity contracts — `SessionKeyManager.sol`, `SpendPolicy.sol`, and `ActionLogger.sol` — and a TypeScript SDK that wraps them. A session key is a `secp256k1` public key granted a scope: a set of target contracts, function selectors, maximum gas per call, maximum spend per call, and an expiry timestamp. The scope is enforced in the contract, not the SDK, so a compromised SDK cannot bypass it.

```
struct SessionKey {
  address[] allowedTargets; // Whitelist; empty = allow all
  bytes4[] allowedSelectors; // Whitelist; empty = allow all
  uint256 maxGasPerCall;
  uint256 maxSpendPerCall;
  uint64 expiry; // Unix timestamp; 0 = no expiry
  bool active;
}

function grantSessionKey(
  address key,
  SessionKey calldata scope
) external onlyOwner {
  require(scope.expiry > block.timestamp, "expiry in past");
  require(scope.maxGasPerCall > 0, "gas cap zero");
  sessionKeys[key] = scope;
  sessionKeys[key].active = true;
  emit SessionKeyGranted(key, scope.expiry);
}
```

The SpendPolicy contract enforces per-action and per-window spend caps. The per-action cap is checked on every call: if the call's msg.value plus any internal transfers exceeds the cap, the call reverts. The per-window cap is checked against a rolling window: the policy stores a spentThisWindow counter and a windowStart timestamp, with automatic window rollover. The ActionLogger contract emits an ActionLogged event unconditionally after every successful call — there is no execution path that produces a state change without a corresponding audit event. This is the structural guarantee that makes agent actions reconstructable from on-chain data alone.

7. Security Architecture & Threat Model

Agent-controlled wallets are a high-blast-radius primitive: a single compromise can drain funds, manipulate state, or cause an agent to take actions its operator never authorised. The security architecture of SigilKit is designed around the assumption that every component will eventually be attacked, and that the contracts themselves must be the last line of defence. This chapter walks through a STRIDE-based threat model, maps each identified threat to a concrete mitigation in the protocol, and specifies the audit tooling stack used to enforce the security posture continuously.

7.1 STRIDE Threat Model

The STRIDE framework — Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege — provides a systematic way to enumerate the attack surface. For each category we identify the specific vectors that apply to agent-controlled wallets and map them to a mitigation in the protocol design. The model treats the agent itself as an untrusted component: even if the agent is compromised or prompt-injected, the contracts should prevent catastrophic loss.

STRIDE Category	Threat Vector (Agent Context)	Mitigation in SigilKit
Spoofing	Attacker forges a session-key signature to impersonate a legitimate agent.	EIP-712 typed-data signatures with domain separator; SessionKeyManager validates key is active + non-expired before any state change.
Tampering	Agent is prompt-injected into calling an unauthorised target with calldata it did not originate.	Session-key scope whitelist (target + selector) enforced in contract; calls outside scope revert at the contract level, not the SDK level.
Repudiation	Agent or operator denies having authorised a past on-chain action.	Every action emits an on-chain ActionLogged event with rationale hash + timestamp; events are immutable and reconstructable from public chain data alone.
Information Disclosure	Agent rationale or sensitive context leaks via calldata or event data.	Only the SHA-256 hash of the rationale is emitted on-chain; the plaintext is stored off-chain by the operator and never published.
Denial of Service	Attacker spams the paymaster with sponsorship requests to drain dApp deposit.	Per-dApp rate limits + per-agent daily gas caps enforced off-chain by the dApp's sponsorship signer; paymaster rejects signatures past the cap.
Elevation of Privilege	Session key attempts an owner-only action (upgrade, revoke, withdraw-all).	Owner-only functions check msg.sender against the stored owner address; session keys can never call them — the function selector is not in any default session-key scope.

Table 7.1 — STRIDE threat model with agent-specific vectors and protocol mitigations.

7.2 Agent-Specific Attack Vectors

Beyond the standard STRIDE categories, agent-controlled wallets face three attack vectors that have no analogue in human-user wallets. **Prompt injection** is the most discussed: an attacker embeds instructions in data the agent reads (a webpage, a chat message, an API response) that cause the agent to issue a malicious transaction. SigilKit's defence is structural rather than algorithmic: even a fully-compromised agent cannot call a target outside its session-key scope, cannot spend past its per-window cap, and cannot execute after its session key expires. The blast radius of a prompt-injection attack is bounded by the session-key scope, not by the wallet balance.

Key-exfiltration is the second vector: an attacker tricks the agent into leaking its session private key, e.g. by including a "debug" instruction that asks the agent to print its key. The mitigation is that session keys are short-lived (default one hour) and scoped (single target), so even a successful exfiltration has a small and time-bounded impact. Operators can also rotate session keys proactively via the SDK's `rotateSessionKey()` function. **Composability attacks** are the third vector: a malicious target contract attempts to re-enter the wallet during a callback, or to call back into the `SpendPolicy` to manipulate the window counter. The mitigation is a `ReentrancyGuard` on every state-changing function, plus the Checks-Effects-Interactions pattern enforced by code review and verified by Echidna property tests.

7.3 Conformance Harness as Security Control

Component 1's conformance harness is itself a security control. By running the same EIP-7702 authorisation flow against viem, ethers, MetaMask, and Coinbase Wallet on every commit, SigilKit detects wallet-side regressions before they reach production. When MetaMask silently rejected EIP-7702 revocations (issue #35520), the bug went undiscovered for weeks because no canonical library existed to test against. With SigilKit's conformance harness in CI, the same regression would be caught on the next commit — not by a user losing funds. This shifts the burden of standards conformance from individual wallet teams to a shared, audited test suite, which is a structural improvement to the entire ecosystem's security posture.

SECURITY INVARIANTS (FORMALLY VERIFIED)

INV-1: For all wallets W and times T , sum of all values transferred out of W via `executeWithSessionKey` within any window of duration D never exceeds `policies[W].perWindowCap`.

INV-2: For all session keys K , if $K.expiry < block.timestamp$, no call signed by K can modify any state in the wallet contract.

INV-3: For all calls C , the `ActionLogged` event is emitted if and only if C 's inner call succeeded. No silent success path exists.

INV-4: Owner-only functions cannot be invoked through `executeWithSessionKey` regardless of session-key scope.

8. Testing & Quality Assurance Strategy

Production-grade smart contracts require a testing strategy that goes well beyond unit tests. SigilKit uses an eight-layer testing approach — the standard seven layers plus a cross-wallet conformance harness unique to Component 1. This chapter describes each layer, the CI/CD pipeline that orchestrates them, and the rationale for the specific tool choices. The overall philosophy is: catch cheap bugs early (unit tests, linting), catch expensive bugs before mainnet (fuzz, invariant, audit), and catch unknown bugs in production (bug bounty, monitoring).

8.1 The Eight Testing Layers

Layer 1 — Unit tests (Foundry). Per-contract tests that verify each public function behaves correctly on representative inputs. Coverage target is 95% line coverage across all four components. Each unit test is fast (under 100ms) and runs on every commit.

Layer 2 — Integration tests (Foundry). End-to-end tests that exercise the full user journey: deploy a wallet, grant a session key, sponsor a transaction, execute it, verify the audit event was emitted. Integration tests use a real ERC-4337 EntryPoint and a local bundler instance.

Layer 3 — Fuzz tests (Foundry --fuzz-runs 10000). Property-based tests where Foundry generates random inputs to a function and asserts an invariant holds. SpendPolicy and SessionKeyManager are the primary fuzz targets because their correctness depends on arbitrary combinations of inputs (any cap, any window, any target list).

Layer 4 — Invariant tests (Foundry --invariant). Higher-level property tests that assert global invariants hold across sequences of calls. Examples: "total spent across any window never exceeds the cap", "no session key can call an owner-only function", "the contract balance equals the sum of deposits minus the sum of withdrawals".

Layer 5 — Static analysis (Slither + solhint). Slither runs symbolic execution and dataflow analysis on the contract source to detect reentrancy, uninitialized storage, shadowing, and other patterns. Solhint enforces style rules that prevent common mistakes. Both run on every commit; new high or medium Slither findings block the merge.

Layer 6 — Property fuzzing (Echidna, nightly). Echidna is a more powerful property-based fuzzer than Foundry's built-in, with smarter input generation and longer run times. It runs nightly on the full contract suite, targeting the same invariants as Layer 4 but with a much larger search budget.

Layer 7 — Fork tests (Foundry --fork-url base). Tests that run against a live Base mainnet fork, using real Uniswap, Chainlink, and USDC contract addresses. Fork tests catch integration bugs with external protocols that local mocks cannot reproduce.

Layer 8 — Cross-wallet conformance harness (Component 1 only). A Foundry + Anvil test suite that exercises the same EIP-7702 authorisation flow against viem, ethers, MetaMask (via Anvil fork), and Coinbase Wallet (via Anvil fork). Asserts byte-for-byte identical behaviour across all four wallets. Catches wallet-side regressions before users do.

Formal verification (Certora, pre-mainnet gate). Beyond the eight runtime testing layers, the most critical invariants — specifically the four security invariants listed in Chapter 7 (INV-1 through INV-4) — are formally verified using Certora Prover before each mainnet release. Certora takes specifications written in the Certora Verification Language (CVL) and proves, via symbolic execution and SMT solving, that the Solidity implementation satisfies the specification across all possible inputs and states. This is a stronger guarantee than fuzz testing: where Echidna and Foundry invariant tests check random sequences, Certora checks the entire state space. The pre-mainnet gate requires all four Certora specifications to pass with zero counterexamples; any counterexample blocks deployment until either the implementation or the specification is revised and re-verified.

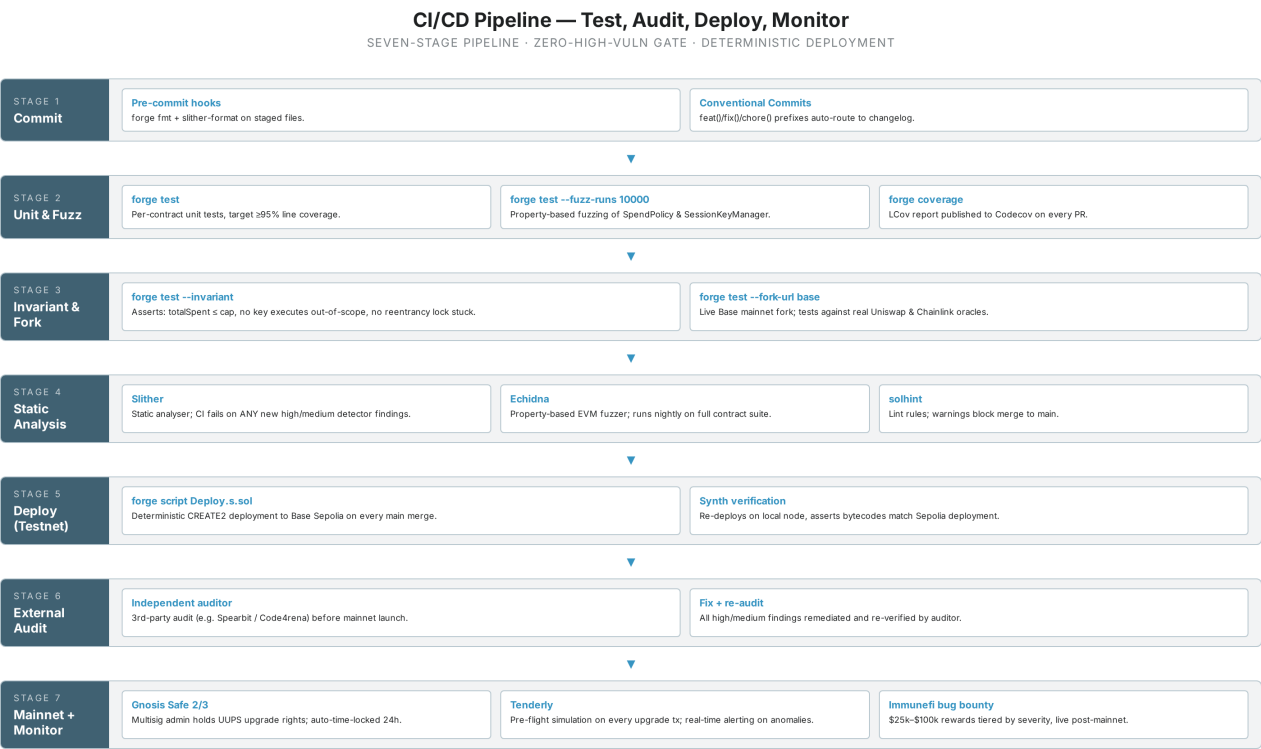


Figure 8.1 — CI/CD pipeline with eight testing stages and mainnet launch gates.

9. Deployment & Operations

Deployment strategy for SigilKit is deliberately conservative: testnet first, deterministic addresses, multisig admin, time-locked upgrades, and an external audit before any mainnet assets are placed under contract control. This chapter documents the deployment plan, the operational tooling, and the runbook for common operational tasks. The guiding principle is that every action on mainnet should be simulable, reversible within 24 hours, and observable in real time.

9.1 Multi-Chain Deployment

SigilKit ships on four chains: Base (primary), Ethereum mainnet, Arbitrum, and Optimism. All four deployments use CREATE2 deterministic deployment, meaning the same deployer address with the same salt produces the same contract address on every chain. This has three operational benefits: developers can hardcode the SigilKit contract addresses in their SDK configuration regardless of which chain they target; the same SDK works on testnet and mainnet without code changes; and address-allowlisting on external contracts needs to be done only once and propagates across chains.

The deployment sequence is staged across three environments. **Base Sepolia** and **Ethereum Sepolia** are the testnet targets — every merge to main deploys to both testnets automatically via the CI/CD pipeline. **Base Mainnet** is the primary production target, deployed after the external audit is complete and the bug bounty has been live for at least two weeks. **Arbitrum** and **Optimism mainnet** are deployed in a second wave, one week after Base mainnet, to allow for staged rollout and monitoring.

9.2 Upgradeability and Admin

All contract-layer components (Components 2 and 4) use the UUPS proxy pattern. Upgrade authority is held by a 2-of-3 Gnosis Safe multisig with three signers: the project lead, an independent security engineer, and a time-locked backup signer that requires 24 hours to execute. Every upgrade transaction is wrapped in a TimelockController that enforces a 24-hour delay between proposal and execution, giving the community time to review and exit if they disagree with the upgrade.

The SDK layer (Components 1 and 3) is versioned via npm and follows semantic versioning. Breaking changes require a major version bump and a migration guide. The conformance harness (Component 1) is run on every commit against the four supported wallets, so wallet-side breaking changes are detected immediately and a compatibility patch can be shipped before downstream users are affected.

9.3 Operations and Observability

Operational tooling is built around three pillars: **Tenderly** for transaction simulation and real-time alerting, **Dune** for public aggregate metrics, and **The Graph** for indexed event data. Tenderly simulates every upgrade transaction before it is signed by the multisig, so signers can see exactly what state changes the upgrade will cause. The Dune dashboard is public and surfaces metrics like total transaction volume through SigilKit wallets, active agent count, policies enforced, and failure rates.

9.4 Deployment Runbook

The runbook below documents the step-by-step procedure for a mainnet deployment of the contract-layer components. Each step has a responsible party, an expected output, and a rollback plan. The runbook is rehearsed end-to-end on Base Sepolia before every mainnet deployment.

Step	Action	Owner	Rollback
1	Confirm audit report is signed and published	Project lead	Delay deployment
2	Verify Immunefi bounty is live and funded	Security eng	Delay deployment
3	Run final CI/CD pipeline on release tag (8 layers)	CI bot	Fix failing stage
4	Run cross-wallet conformance harness on release tag	Security eng	Fix conformance failure
5	Simulate deploy script on Tenderly fork	Project lead	Fix script
6	Propose upgrade via Gnosis Safe (2-of-3 signatures)	Multisig signers	Cancel proposal
7	Wait 24h time-lock to expire	Auto	Cancel within 24h
8	Execute deployment transaction on Base mainnet	Multisig signers	Pause contract
9	Verify deployed bytecodes match audited source	Security eng	Pause + redeploy
10	Cross-chain deploy to Arbitrum + Optimism (T+7 days)	Project lead	Delay cross-chain
11	Announce deployment on Twitter + Discord	Project lead	N/A
12	Enable Tenderly alerts; monitor first 24h closely	On-call eng	Pause if anomaly

Table 9.1 — Mainnet deployment runbook with rollback plans.

10. Roadmap & Milestones

The SigilKit roadmap is structured as three horizons over twelve weeks. Each horizon has a single dominant deliverable, a small number of supporting deliverables, and a clear success metric. The structure is deliberately conservative: no horizon starts until the previous horizon's success metric is met, and no mainnet deployment happens until the external audit is complete. Because each component is independently shippable, the roadmap can be parallelised across multiple engineers once Horizon 1 is complete.

10.1 Horizon 1 — Component Foundations (Weeks 1-4)

The first horizon ships the four components at production-grade quality with full Foundry test suites and Slither clean. The dominant deliverable is a working npm install `@sigilkit/sdk` that exports all four components, plus a deployed-and-verified instance of the contract-layer components on Base Sepolia. The success metric is: 95%+ line coverage on all four components, all invariant tests passing at 10,000 runs, zero Slither high or medium findings, the cross-wallet conformance harness green across viem/ethers/MetaMask/Coinbase, and Base Sepolia deployment live.

Week-by-week breakdown: **Week 1** — repository scaffolding, Foundry + viem setup, Component 3 (Multi-RPC Provider) shipped first as it is the most independent and has the clearest acceptance criteria (closes ethers #1053). **Week 2** — Component 2 (Diamonds Module) shipped with full Echidna property tests. **Week 3** — Component 1 (EIP-7702 Wallet Library) shipped with conformance harness wired into CI. **Week 4** — Component 4 (Agent Session-Key Manager) shipped with full invariant tests, integration tests across all four components, Base Sepolia deployment live.

10.2 Horizon 2 — Reference dApp + Demo Agent (Weeks 5-8)

The second horizon adds a reference dApp and a demo agent that exercises all four components end-to-end. The dominant deliverable is a published npm package `@sigilkit/sdk` at v1.0-rc, plus an open-source reference dApp that lets a developer create an agent wallet, grant a session key, and execute a sponsored transaction in under five minutes. The demo agent is an autonomous USDC treasury manager that executes limit orders via UniswapX based on a configurable strategy.

Week-by-week breakdown: **Week 5** — SDK packaging, npm publish as next tag, reference dApp scaffolding. **Week 6** — demo agent implementation (treasury manager), README and getting-started guide. **Week 7** — integration tests against Base Sepolia, gas optimisation pass, documentation website. **Week 8** — external review by 3 early-adopter projects (Sphere SDK, Helix, Sandboxed.sh contacted), feedback incorporated, v1.0-rc published.

10.3 Horizon 3 — Audit, Bounty, Mainnet (Weeks 9-12)

The third horizon executes the external audit, opens the bug bounty, and deploys to Base mainnet. The dominant deliverable is a mainnet deployment of the audited contract components under a 2-of-3 Gnosis Safe admin, with an Immunefi bug bounty live for at least two weeks prior. The success metric is: audit report published with all high and medium findings remediated, bug bounty live with at least \$50k in maximum rewards, Base mainnet deployment live and stable for 7 days, and at least one external dApp (Sphere SDK, Helix, or Sandboxed.sh) integrated with SigilKit in production.

Week-by-week breakdown: **Week 9** — submit contracts to external auditor (Spearbit or Code4rena competitive audit), open Immunefi bug bounty scope. **Week 10** — receive audit report, remediate findings, re-audit if needed. **Week 11** — final CI/CD pipeline green, Tenderly monitoring configured, deployment runbook rehearsed on Base Sepolia. **Week 12** — Base mainnet deployment, cross-chain to Arbitrum + Optimism (T+7), public announcement, grant applications submitted to Base ecosystem and Arbitrum Stylus Sprint.

10.4 Grant and Funding Strategy

SigilKit is engineered to qualify for ecosystem grant funding from four active programs. The **Ethereum Foundation Mission Request** for EIP-7702 tooling is the most direct fit — Component 1 closes this request directly. The **Base ecosystem grant program** funds projects in the \$20K-\$250K range that drive Base adoption; SigilKit's SDK-first developer onboarding and Base-primary deployment make it a strong fit. The **Arbitrum Stylus Sprint** has a 5M ARB pool; while SigilKit is Solidity-first, a Rust SDK wrapper for the Stylus ecosystem is a natural extension. **Optimism Retroactive Public Goods Funding** (RetroPGF) is a longer-term play: by publishing impact metrics on the public Dune dashboard, SigilKit becomes a strong candidate for retroactive grants once the toolkit has demonstrable adoption.

Horizon	Weeks	Dominant Deliverable	Success Metric	Funding Trigger
1 — Foundations	1-4	4 components + Foundry tests + harness	95% cov, 0 Slither high, harness green	EF Mission Request submitted
2 — SDK + Demo	5-8	npm package + reference dApp + demo agent	<5min to first agent tx	Base grant application
3 — Audit + Main	9-12	Mainnet deployment + Immunefi bounty	7-day stable mainnet, 1 ext dApp	Base + Arbitrum + RetroPGF

Table 10.1 — Three-horizon roadmap with success metrics and funding triggers.

The twelve-week timeline is aggressive but achievable for a senior engineer working full-time. The critical path is the external audit in Horizon 3 — audit firms typically have 4-8 week backlogs, so the audit engagement should be booked at the start of Horizon 2 to ensure the audit slot is available when Horizon 3 begins. If the audit slips, the mainnet deployment slips; there is no path to mainnet that bypasses the audit gate.